

Chapter 6: CPU scheduling

Scheduling criteria

always keep CPU busy

- ① CPU utilization (least idle time) $\uparrow$
- ② Throughput (Rate of process completion)
- ③ Turnaround Time (time to complete a process)
- ④ Waiting Time (in the ready queue) $\rightarrow$ waiting time minimum
- ⑤ Response Time (between submit & response)

scheduling optimization

- ① Max CPU utilization
- ② Max Throughput
- ③ Min Turnaround Time
- ④ Min waiting Time
- ⑤ Min Response time

FCFS (First come, First served)

The processes P_1, P_2, P_3 arrive at time 0

Process time

P_1

P_2

P_3

24

3

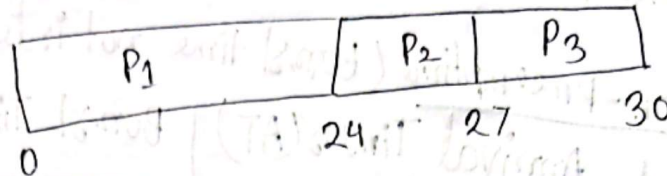
3

3

charat can come for xm

Ready Queue (RQ) Processing

$P_3 \quad P_2 \quad P_1$
 $P_1 \quad P_3 \quad P_2$



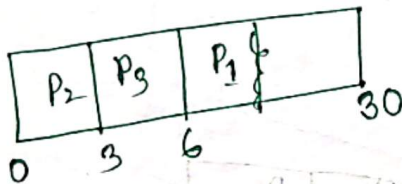
waiting time (In the RQ)

$$P_1 = 0$$

$$P_2 = 24$$

$$P_3 = 27$$

$$\text{Avg. waiting time (WT)} = \frac{0 + 24 + 27}{3} = 17$$



waiting time: $P_1 = 6$

$$P_2 = 0$$

$$P_3 = 3$$

$$\text{Avg. WT} = \frac{6 + 3}{3}$$

$$= 3$$

average waiting time changes
 because of order also convoy
 effect

FCFS WT and RT is always
 same

* criteria "Arrival time"

* Mode "Non-preemptive"

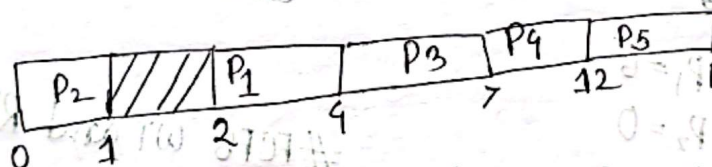
xm-convey effect why happen?
 → convey effect happened in FCFS because processes are executed in the order they arrive. if long process comes first, it keeps the CPU for a long time, so all shorter processes must wait behind. that's why convey effect occurs.

- Short (burst) Processes behind Long (burst) Processes
- e.g I/O bound processes behind a CPU bound process

FCFS → Non-preemptive (Burst time not interrupt)

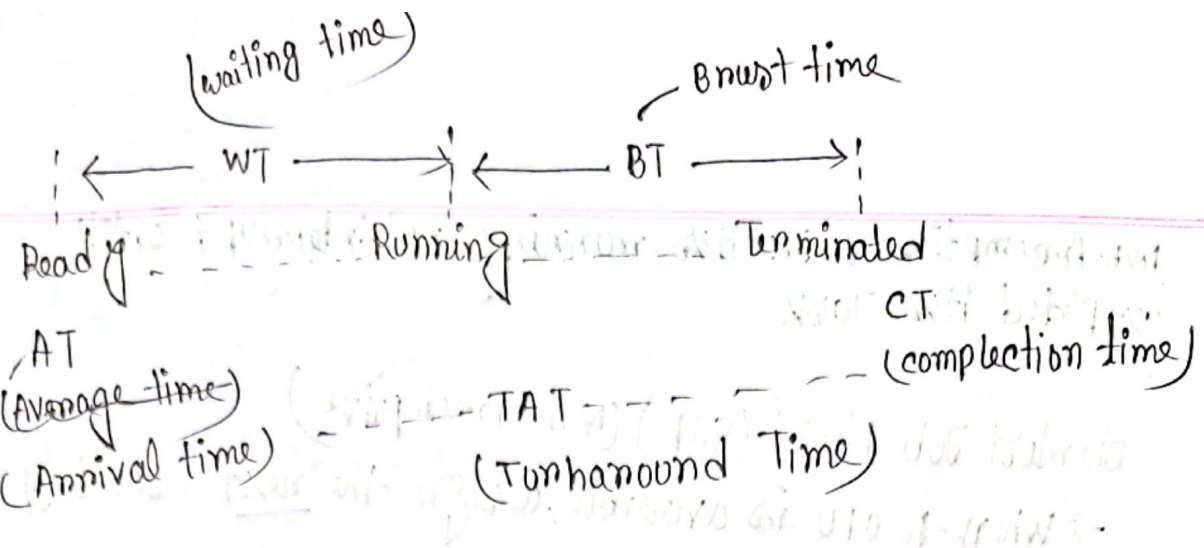
# Processes	Arrival Times (AT)	Burst Times
P ₁	2	2
P ₂	0	1
P ₃	2	3
P ₄	3	5
P ₅	4	4

RG P₅ P₄ P₃ P₁ P₂



Non-preemptive - A scheduling method once a process gets the CPU, it runs until it finishes and can't be interrupted by another process.

Preemptive - A scheduling method where the CPU can be taken from running process and given to another process before it finishes.



$$TAT = CT - AT$$

$$TAT = WT + BT$$

$$WT = TAT - BT$$

CT time check from chart ←

P	CT	AT	TAT	BT	WT
P ₁	4	2	4-2=2	2	2-2=0
P ₂	1	0	1-0=1	1	1-1=0
P ₃	7	2	7-2=5	3	5-3=2
P ₄	12	3	12-3=9	5	9-5=4
P ₅	16	4	16-4=12	4	12-4=8

$$Avg\ WT = \frac{0+0+2+4+8}{5} = 2.8$$

$$Avg\ TAT = \frac{2+1+5+9+12}{5} = 5.8$$

TA = burst + WT
 (TA) burst + WT
 (TA) burst + WT
 (TA) burst + WT

Non-Preemptive when it's running not interrupt until completed it's Task

Shortest Job First (SJF) (Non-Preemptive)

→ When the CPU is available, assign the next shortest burst time.

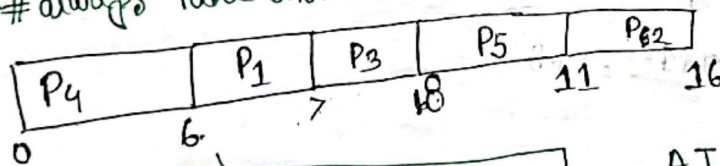
→ If there is a tie use FCFS

RG = P3 P5 P4 P1

→ For this order always use (AT)

Process	AT	BT	CT	TAT	WT	RT
P ₁	2	1	7	5	4	6 - 2 = 4
P ₂	1	5	16	15	10	16 - 1 = 15
P ₃	4	1	8	4	3	8 - 4 = 4
P ₄	0	6	6	6	0	6 - 0 = 6
P ₅	2	3	11	9	6	11 - 2 = 9

always take shortest arrival time



SJF → Min. WT

Response time (RT)

When the CPU was allocated for the first time after arrival.

$$RT = F_{\text{first}} - AT$$

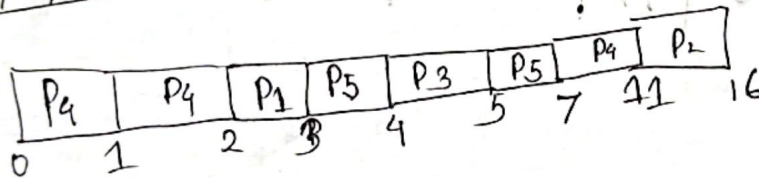
(First touched by CT)
From left side

→ whenever new process arrives, there may be pre-emption of the running process

SJF + Preemption → SRTF
(Shortest Remaining time First)

P	AT	BT	CT	TAT	WT	RT
P ₁	2	4	3	1	0	0
P ₂	1	5	16	15	10	10
P ₃	4	4	5	1	0	0
P ₄	0	4	11	11	5	0
P ₅	2	2	7	5	2	1

RQ
P₅ P₃ P₄ P₅ P₁ P₂ P₄



$$\text{avg. TAT} = \frac{33}{5} = 6.6$$

$$\text{avg WT} = \frac{17}{5} = 3.4$$

SJTF give minimal average waiting time
→ we haven't get average WT. less than 3.4 that's why it's optimal solution give minimal average.

SJF (Shortest Job first / Shortest Job next)
→ Out of all available (waiting) process it selects the process with the smallest burst time to execute next.

TQ ↓ CT ↑

24-2-26

→ FCFS + Preemption

→ Round Robin

→ Time Slice / time Quantum

* Best Response Time

Time Quantum: Allowed time to run in the CPU without breaks

Time Quantum, TQ = 3

Processed	AT	BT	CT	TAT	WT	RT
P ₁	0	8 20	22	22	14	0
P ₂	5	20	11	6	4	4
P ₃	1	7 10	23	22	15	2
P ₄	6	30	14	8	2	5
P ₅	8	3 20	25	17	12	9

RQ

P₅ P₃ P₅ P₃ P₄ P₂ P₁ P₃ P₁

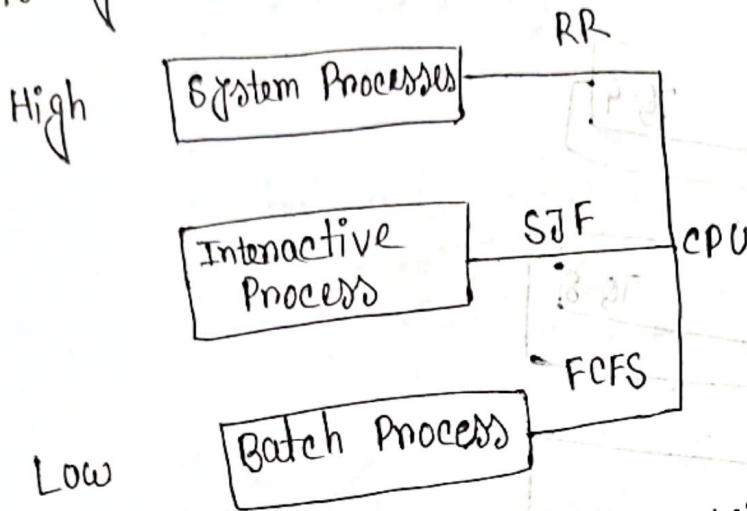
P ₁	P ₃	P ₁	P ₂	P ₄	P ₃	P ₅	P ₁	P ₃	P ₅
0	3	6	9	11	14	17	20	22	23 25

Find Average time

Multi level Queue

- Processes are different by nature.
- should use different Ready Queues

Priority



multi level disadvantage System Processes and Interactive Process coming but never get chance Batch Process.

Problem → starvation: High priority queue processes will always get CPU first.

Solution → Aging: AS time progresses, increase the priority of the processes.

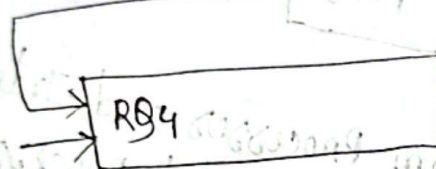
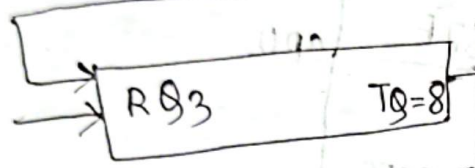
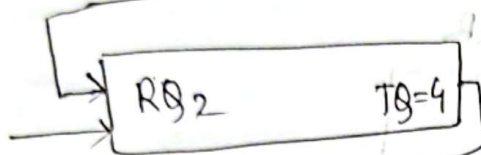
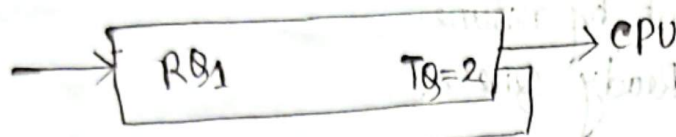
Multi level Feedback Queue:



theoretical questions for xm

quiz → 8 tomik
Threads & CPU
scheduling

Multi level Feedback Queue:



period

S (System Times): Time period expires, Promote the lowest priority processes to the top.

cpu scheduling algorithm, processes placed in multiple queues with different priorities. A process can move between queues based on their behavior. high priority queue get first CPU.

1-3-26

1-3-26

chapter - 6 : Synchronization Tools

synchronization Tools:

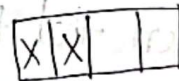
- concurrent execution on parallel execution can contribute to issues involving integrity of data shared by several process.
- our bounded-buffer in shared memory could be taken as an example
- Our bounded buffer allowed at most $\text{BUFFER_SIZE} - 1$ items
- if we modify the algorithm by adding count variable

Producer

```

while(1) {
    /* Produce an item in
       next - produced */
    while (count == BUFFER_SIZE) {
        /* do nothing */
    }
    buffer[in] == next - produced;
    in = (in + 1) % BUFFER_SIZE;
    count++;
}

```



consumer

```
while(1) {
```

```
    while(count == 0) {
```

```
        /* do nothing */ → close on stuck until  
        fulfilled the condition
```

```
    }
```

```
    next_consumed = buffer[out];
```

```
    out = (out + 1) % BUFFER_SIZE;
```

```
    count --;
```

```
}
```

→ Although, the Producer & consumer routines/functions are connected separately, they might not function correctly when executed concurrently.

e.g. → consider that the value of count (shared variable) is currently 5.

→ The Producer executes count++
& The consumer " " --

Logically one after the other

→ Following the execution order of the 2 statements, the value of count may be 4, 5 or 6

count ++

register₁ = count

register₁ = register₁ + 1

count = register₁

→ Producer (P)

count --

register₂ = count

register₂ = register₂ - 1

count = register₂

→ consumer (C)

concurrent execution of count ++ & count --
→ occur only one shared value time

Time → Producer

T₀ : P register₁ = count [register₁ = 5]

T₁ : P register₁ = register₁ + 1 [register₁ = 6]

T₂ : C register₂ = count [register₂ = 5]

T₃ : C register₂ = count - 1 [register₂ = 5]

T₄ : P count = register₁ [count = 6]

T₅ : C count = register₂ [count = 4]

exit

* A situation like this, where several processes access & manipulate the same data concurrently & the outcome of the execution depends on the order, it is called a Race condition.

→ We need synchronization to mitigate Race conditions.

Critical Section Problem

→ A system of n processes $\{P_0, P_1, P_2, \dots, P_n\}$ each process has a critical section.

→ A critical section (CS) is a segment of code where processes are changing common variables, writing to files etc.

→ When one process is in its CS, ~~no other process is in its CS~~, no other process can be in their CS.

Race condition Mitigation

condition

Mutual Exclusion

Meaning =

only one process can be in their CS at a time.

Progress

if no process is in their CS, and one process wants to enter its CS, that process must be allowed to proceed

Bounded waiting

A process waiting to enter its CS will eventually get a turn no infinite waiting.

* The CS Problem could be solved simply a single-processor environment

→ Prevent interrupts/context switch from happening while a shared variable is being modified

* Unfortunately this solution is not feasible in a multiprocessor environment

→ Disabling Interrupts is time consuming since message needs to be passed to all processors.

peterson's solution

→ A classic software based solution to the CS Problem.

* May not work correctly on modern computers.

→ Good Algorithmic description that addresses Mutual Exclusion, Program progress Bounded waiting

→ peterson's solution is restricted to only 2 processes that alternate execution between their critical sections & Remainder section.

Peterson's Solution

- Peterson's solution is restricted to 2 processes

Let the processes be P_1 & P_2 2 data items to be shared

while (1) {

Enter

CS

Exit

RS (Remainder section)

P_1 P_2
flag 0/1 0/1

int turn

0/1, indicating which
turn it is to enter CS.

→ Humble Algorithm

P_i

while (1) {

flag[i] = 1;

turn = i;

while (flag[i] & turn == i) { }

/* critical section */

flag[i] = 0;

/* Remainder

section */

}

boolean flag [2]
int

used to indicate if a process is
ready to enter its CS

stuck/spinlock

P_j

while (1) {

flag[i] = 1;

turn = i;

while (flag[i] && turn == i) { }

/* CS */

flag[i] = 0;

/* RS */

}

P_i ← 0

① flag[i] = 1;

③ turn = i;

↳ shared variable

⑤ while (flag[i] && turn == i);

final value of turn = i

0 → because turn = i

⑦ critical section

⑨ flag[i] = 0;

⑩ RS

P_j ← 1

② flag[i] = 1;

④ turn = i;

⑥ while (flag[i] && turn == i) { }

⑧ stuck in spinlock, waiting

⑪ while (flag[i] && turn == i);

⑫ CS

⑬ flag[i] = 0

⑭ RS

① mutual exclusion

→ only one process is able to enter it's CS at a time

time

② Progress $\rightarrow$ If one process is not ready & the other process is ready, the other process is allowed to enter or proceed to its CS.

P_i
flag[i] = 0

This process executed RS
next $L \rightarrow 69$

P₂
flag[j] = 1

turn = 1;

while (flag[i] == 1) {

$$\frac{+unn = +i}{1}$$

* Enten's CS

③ Bounded waiting

Bounded waiting
 ↳ Since turn variable gives priority to the other process, each process waiting a maximum of 1 time

turn.

p_i : Remainder

Pr. 11

wait
when to enter CS | CS | Reminders
waits | CS

peterson's solution mitigates race condition.

Lock Mechanism:

- Multi-Process solution for the critical section problem
- executes in user mode
- Doesn't guarantee Mutual exclusion

Typical structure of a process

```
while(1) {
```

ENTRY

critical section

EXIT

Remainder section

```
}
```

Every process will follow this format

```
do {
```

Acquire lock ENTRY

CS

Release Lock EXIT

```
}
```

P₅ Lock $\rightarrow$ 0/1

CS

$P_0, P_1, P_2, \dots, P_n$

Lock Mechanism:

- Multi-Process solution for the critical section problem
- executes in user mode
- Doesn't guarantee Mutual exclusion

Typical structure of a Process

```
while(1) {
```

```
    ENTRY
```

```
    critical section
```

```
    EXIT
```

```
    Remainder section
```

```
}
```

Every process will follow this format

```
do {
```

```
    Acquire Lock ENTRY
```

```
    CS
```

```
    Release Lock EXIT
```

```
}
```

```
[P5] Lock → 0/1
```

```
CS
```

P₀, P₁, P₂, ..., P_n

LOCK Mechanism

```

while(1) {
    ① while (lock == 1) // check if CS is empty. Else stay in spin lock
    ② lock = 1; // Lock before entering CS
}

```

/*critical section*/ (CS)

Exit [③] lock = 0 // unlock when exiting CS

1* Remainder Section* / (RS)

}

case-1

P_0

P1

lock

CS

1

2

48

CS & Preemption

3

RS *

RS -

1.

 1^*

- 2 -

$$1, x$$

-2

— 06, *

—CG,*

CS

9

-1

Q

2

1

4

2

P.

Po

97

—

1

time

Case-2

P ₀	P ₁	Lock	CS
1, *		0	
1, *	1	0	
1, *	2	1	
1, *	CS	1	P ₁
2	CS, *	1	
CS	CS, *		P ₁ , P ₀

```

while (1) {
    ① while (lock == 1),
    ② lock = 1 ;
        /* CS */
    ③ lock = 0
        /* RS */
}
    
```

This happens because of preemption between ① & ②